

多模态稀疏 Attention 与定制 Mask Kernel

从论文图示、核心机制到 mask lowering、kernel metadata 与 host-device 数据流

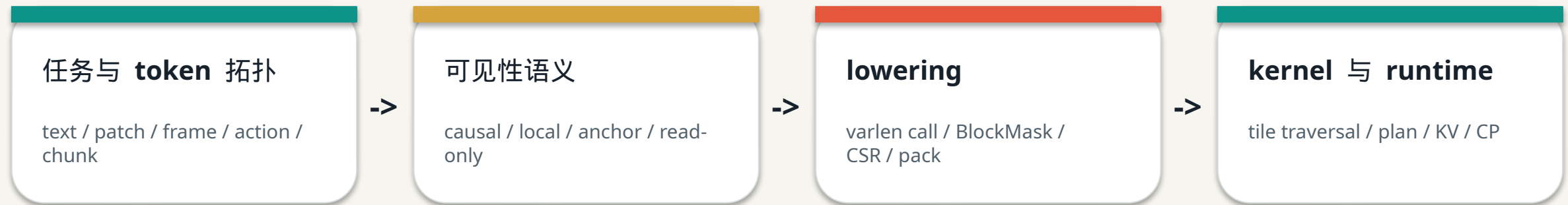
理解 high-resolution VLM token selection

统一 two-way stream lowering / special causal

生成 window, CSR, router, temporal reuse

如何读本报告：一张图必须落到一个可执行对象

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点



每篇均回答：解决什么问题？图中 **token/block** 是什么？**mask** 怎样表示？**kernel** 到底跳过了什么？哪部分只有论文证据？

读图规则

论文图说明算法语义；源码路径说明 runtime 真正接收什么；性能图只能支持其报告设置，不能自动归因给单个 kernel。

领域图谱：mask 不再只是一个 score bias

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

规则型	Causal-rCM / Cosmos 3	block schedule, stream split	BlockMask / varlen calls
索引型	LVSA	CSR indptr + indices	FlashInfer planning
选择型	VMoBA / TSA / VLM FlexAttn	index set + compact QKV	FlashAttention varlen
替代型	FrameDiT	frame-level matrix representation	不生成 token-pair mask

系统趋势：将“可见性”保留为规则、索引或紧凑序列，直到 **kernel plan**；禁止在模型脚本侧先 **materialize L x L**。

理解例：FlexAttention VLM 只让 query 读取被选中的高分辨率细节

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

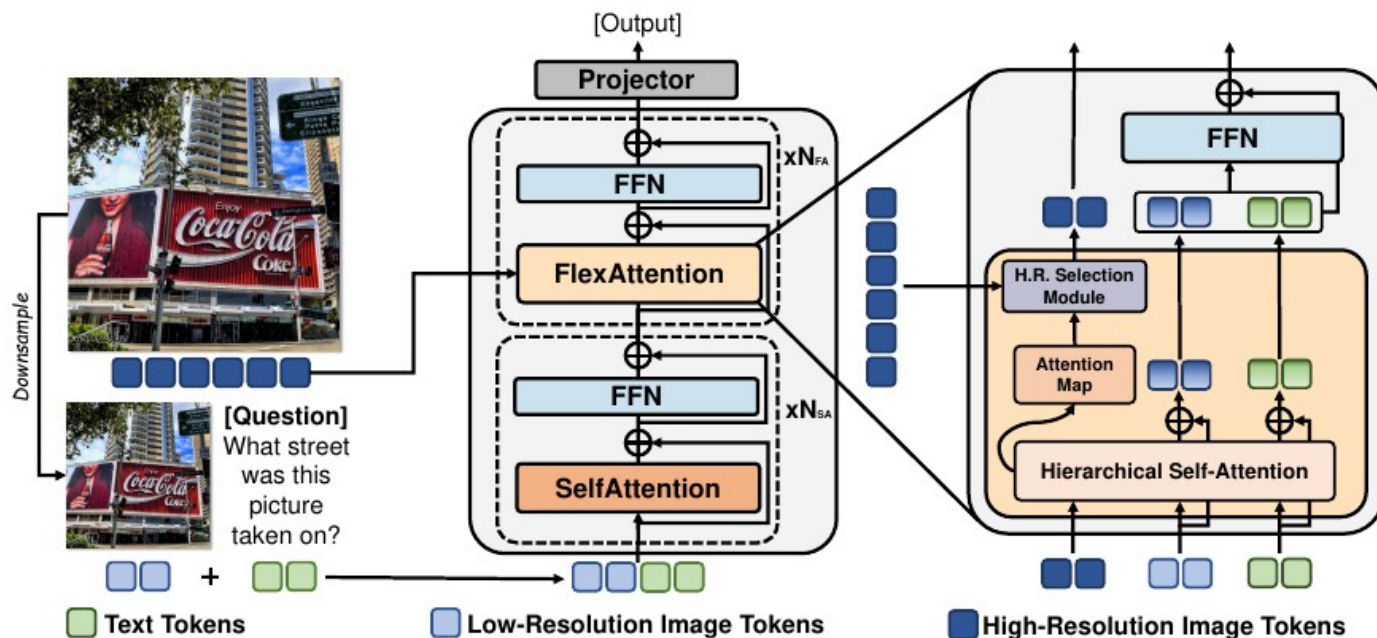


Fig. 2: An Overview of FlexAttention. Within each FlexAttention layer, the encoded high-resolution image features are selected according to the input attention map. These selected features are then inputted into the hierarchical self-attention mechanism alongside input hidden states, which encompass both low-resolution image tokens and text tokens, for computation.

解决的问题

全量 high-resolution patch 进入 decoder 会产生 quadratic query/key work；只降采样又丢小文字和小目标。

图中机制

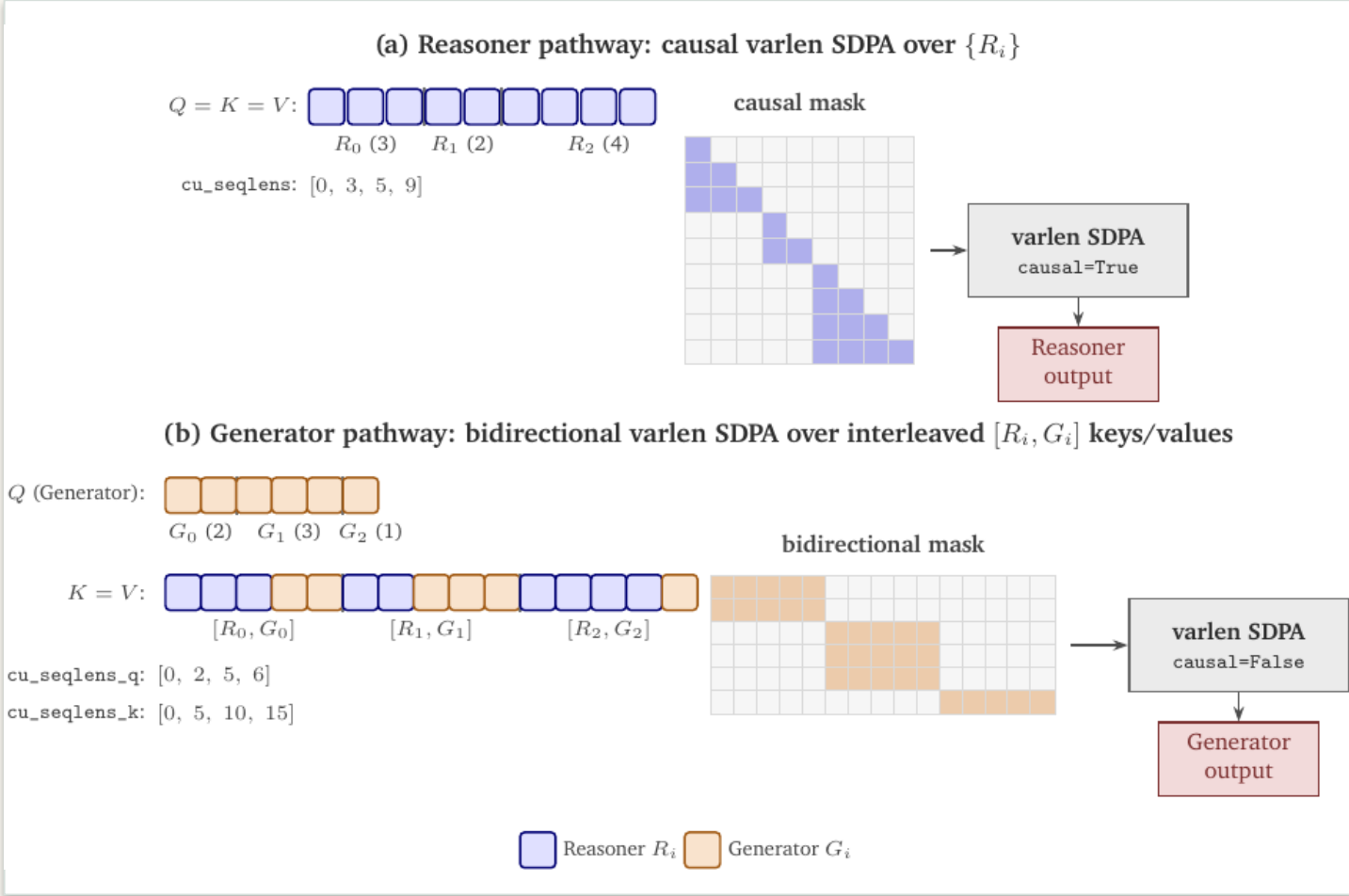
低分辨率与文本保留 N-token residual stream；上一层 attention map 选择约 10% high-resolution features，仅作为额外 K/V。

实现 / kernel 含义

数学对象是 $N \times (N+M)$ rectangular attention。compact/varlen 是实现建议；论文未声称 PyTorch FlexAttention 或特定 sparse CUDA backend。V100 上 TextVQA 总时间比 HD/XAttn 低 13%/24%。

统一模型：Cosmos 3 先拆语义，再复用两类 varlen attention

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点



解决的问题

reasoner 必须 causal 且不能读取 noisy generator ;
generator 又要读取同样本 AR 条件与全部 DM tokens 。

图中机制

同一 packed sample lower 为 AR causal call 与 DM full rectangular call ; 跨样本用 offsets 隔离。

实现 / kernel 含义

收益属于 semantic lowering + FA3/varlen/runtime 组合 ,
不改变模型候选质量。Nano/Super 训练 MFU 仅
0.23/0.30 , 说明 attention 之外仍有 loader、VAE、通
信与 checkpoint 成本。

流式世界模型：Causal-rCM 的 special causal mask

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

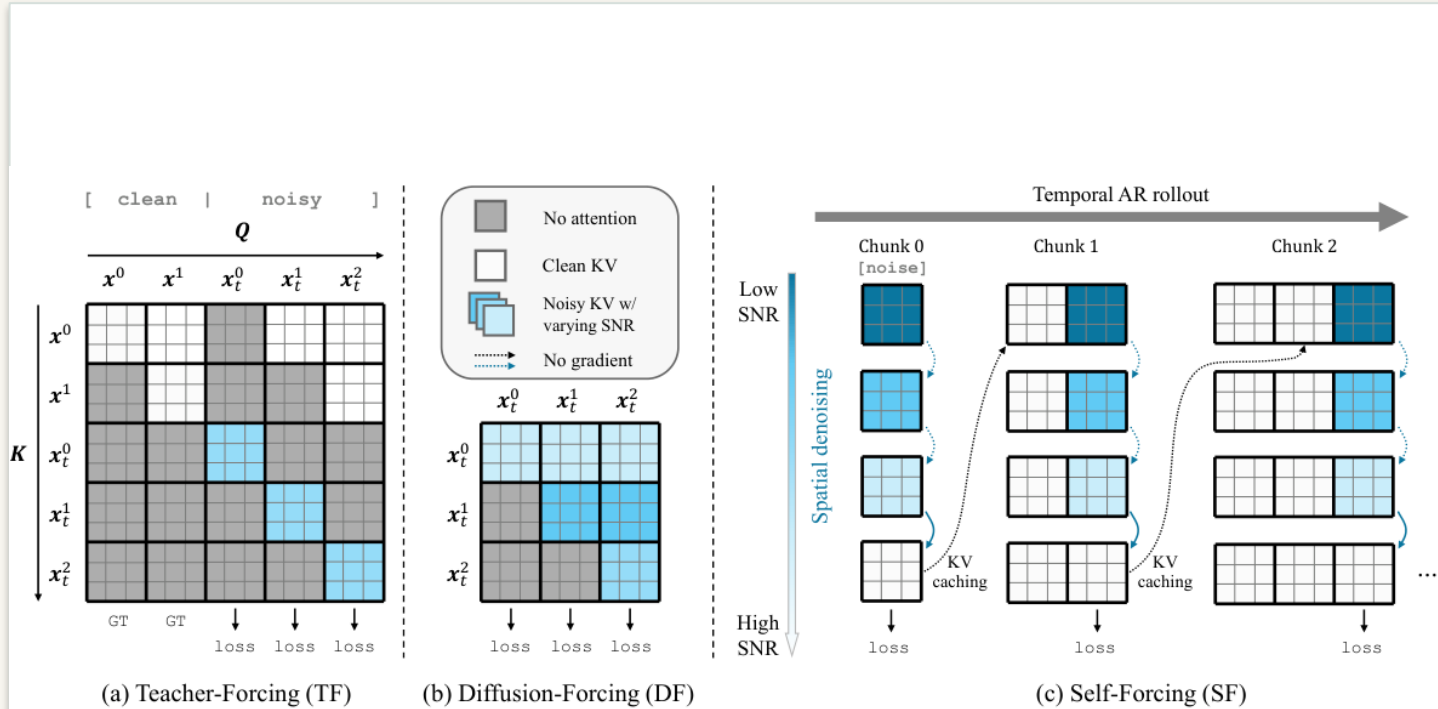


Figure 3: Illustration of causal training paradigms, adapted from Self-Forcing (Huang et al., 2025).

解决的问题

AR diffusion 的 TF 需要 clean history + noisy target ; SF 又需 self-generated rollout 与 KV cache。普通三角 causal mask 不够。

图中机制

图中 TF / DF / SF 区分 clean、noisy、self-generated history ; noisy block 只读允许的 clean history 与自身 block。

实现 / kernel 含义

BlockPattern + AttnMaskSpec -> Flex BlockMask / range metadata ; 同一 mask 进入 primal 和 JVP Triton kernel。Magi backward 限制需单独核验。

长视频：LVSA 用 window + rotating anchors 保持稀疏预算

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

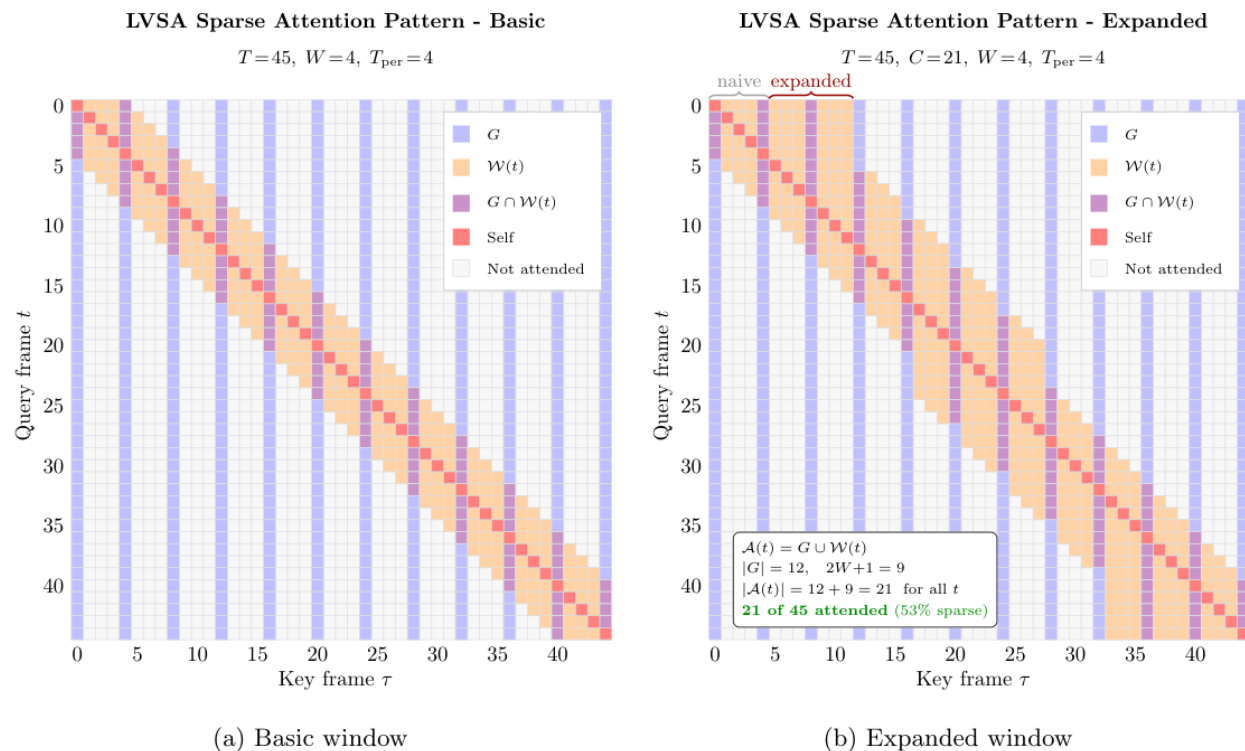


Figure 1: Basic versus expanded window pattern. The basic adaptive window (a) wastes attention budget when the local window overlaps global frames, leaving the per-query attended set below the target C . Expanded bounds (b) account for this overlap by extending the window when needed, so every query frame attends to $|A(f)| = |G| + \min(2W + 1, T - |G|) \approx C$ unique frames.

解决的问题

固定 window 会漏长程依赖；window 与 global anchor 重叠又浪费固定 attention budget。

图中机制

expanded local window 与 periodic global frames 构成 $A(t)=G \cup W(t)$ ，每个 query frame 近似保持相同 attended set 大小。

实现 / kernel 含义

frame-block CSR int32 indptr/indices；FlashInfer BlockSparseAttentionWrapper 跳过未列 tile。CPU planner 留 metadata 在 host，并非 GPU kernel 直接读 CPU RAM。

学习式 block router : VMoBA 的 partition -> select -> varlen

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

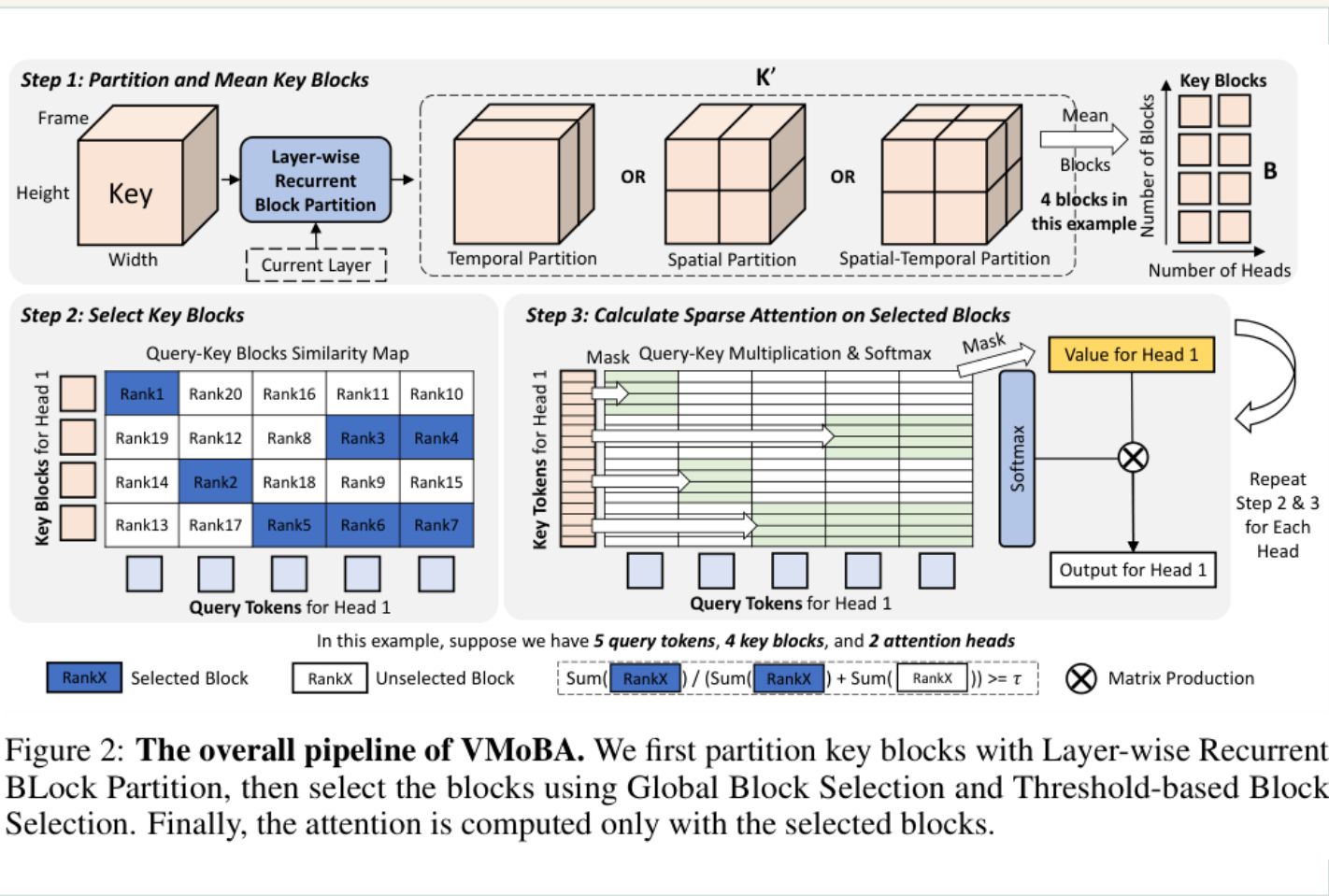


Figure 2: **The overall pipeline of VMoBA.** We first partition key blocks with Layer-wise Recurrent BLock Partition, then select the blocks using Global Block Selection and Threshold-based Block Selection. Finally, the attention is computed only with the selected blocks.

解决的问题

一维均匀 MoBA block 与视频 temporal/spatial/3D 邻域不匹配；固定 top-k 也浪费异质 head 预算。

图中机制

recurrent 1D/2D/3D partition，mean key 产生 block score；global/threshold selection 后仅在选中 blocks attention。

实现 / kernel 含义

GPU gate + topk/threshold -> nonzero -> gather QKV -> cu_seqlens -> FlashAttention varlen。控制面为 gate/sort/pack/LSE，不传 CSR 或 pair-mask。

动态控制面：HASTE 的 mask reuse 与 head-wise calibration

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

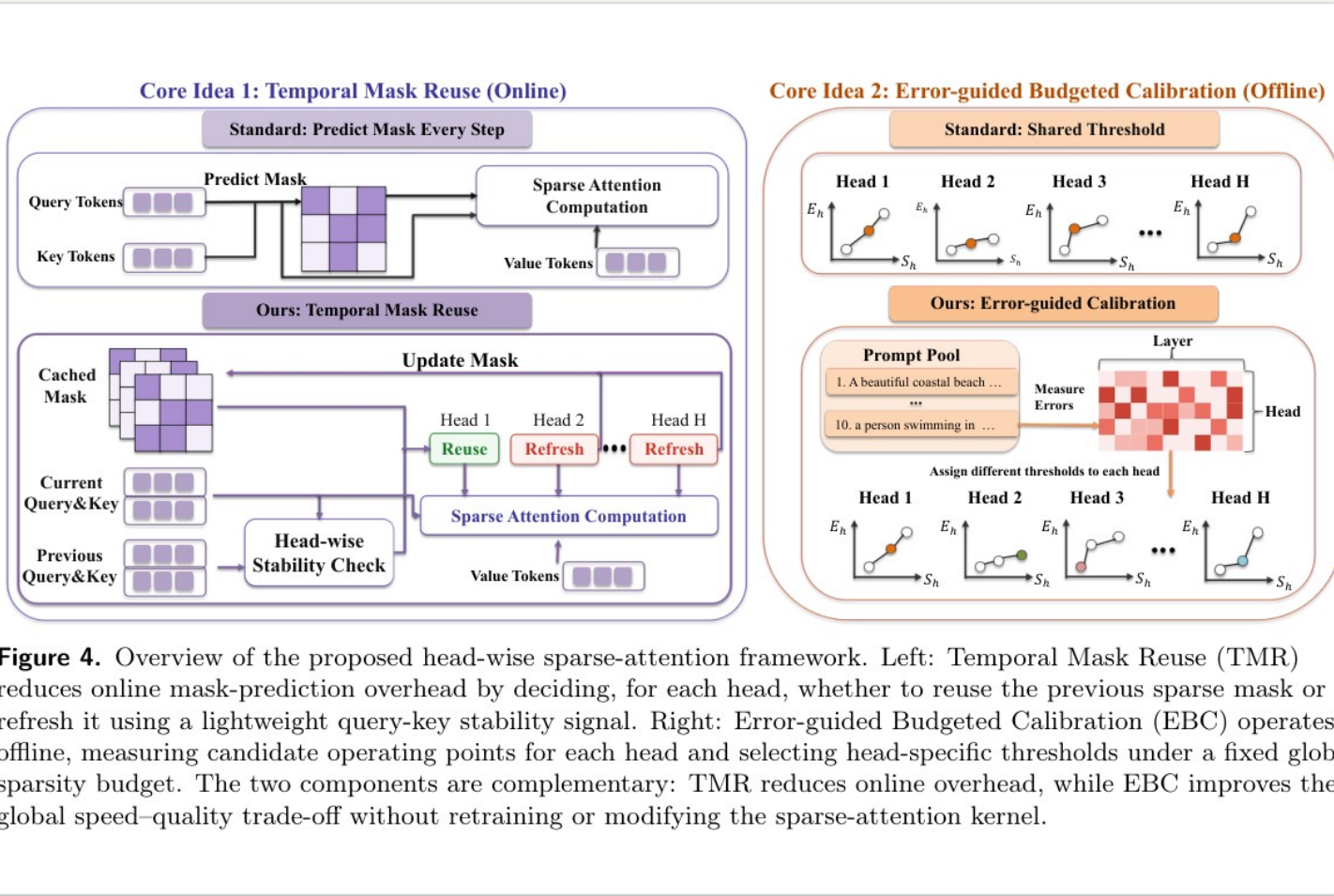


Figure 4. Overview of the proposed head-wise sparse-attention framework. Left: Temporal Mask Reuse (TMR) reduces online mask-prediction overhead by deciding, for each head, whether to reuse the previous sparse mask or refresh it using a lightweight query-key stability signal. Right: Error-guided Budgeted Calibration (EBC) operates offline, measuring candidate operating points for each head and selecting head-specific thresholds under a fixed global sparsity budget. The two components are complementary: TMR reduces online overhead, while EBC improves the global speed-quality trade-off without retraining or modifying the sparse-attention kernel.

解决的问题

Video DiT 要多 step denoise ; 逐 head、逐 step 重算 sparse mask 可能吃掉 attention 节省, 统一 threshold 又不公平。

图中机制

TMR 用 Q/K drift 判断每个 head 复用还是刷新 cached descriptor ; EBC 用离线 error curve 分配 head-specific threshold。

实现 / kernel 含义

应缓存 sparse descriptor , 而非 $N \times N$ mask 。官方代码未取得, CSR/BlockMask/host-device placement 不可断言 ; 机制与结果为 PDF-only 。

Training-free 视频：Sparse VideoGen 的 spatial / temporal head dispatch

图文阅读版 2025-07-16 SUNMILK@CUA 重点

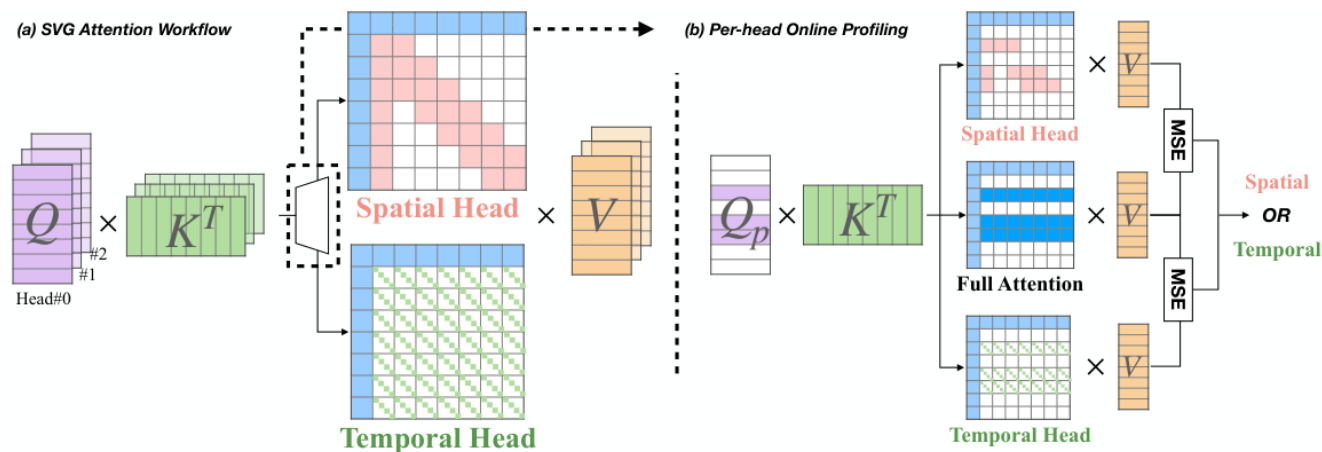


Figure 4. Overview of SVG framework. (a) During generation, SVG adaptively classifies each attention head as either a *spatial head* or a *temporal head* and applies a dedicated sparse attention computation accordingly. (b) This adaptive classification is driven by online profiling strategy, which extracts a small portion of Q , denoted as Q_p , to perform both spatial and temporal attention computations. SVG then selects the attention pattern that yields the minimal MSE compared to full attention, ensuring accurate classification.

解决的问题

Video DiT 3D full attention 成为主成本；直接移植 LLM mask 会丢 temporal dependency。

图中机制

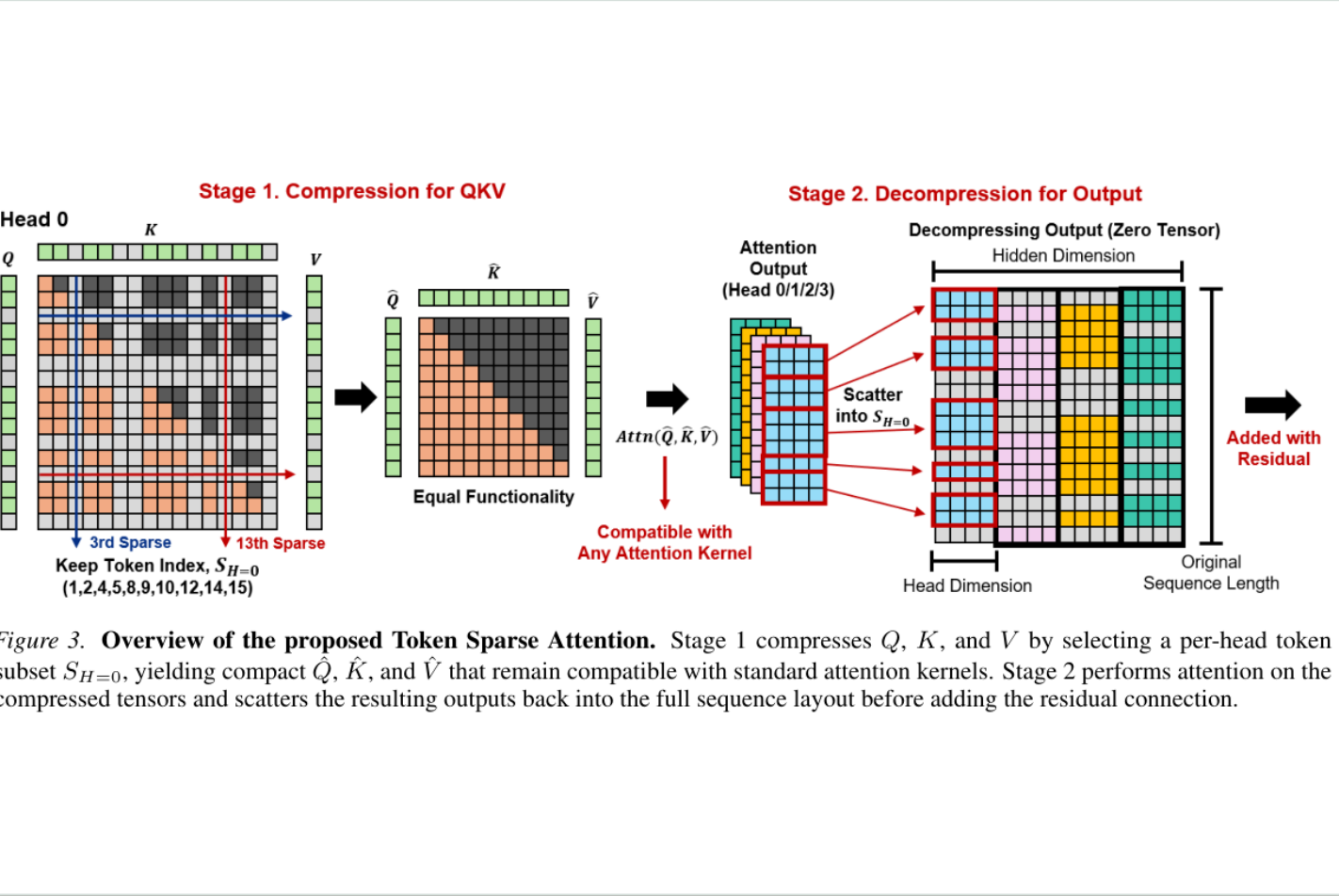
sampled rows 对 spatial / temporal / full attention 做 MSE 近似，按 head dispatch 到专用 pattern。

实现 / kernel 含义

关键不只是找 pattern：temporal slash 必须 layout transform 才能有 coalesced tile access。论文称 Triton/FlashInfer prototype，具体 metadata 本次未有源码核验。

Token Sparse Attention : 保留 selector 灵活性 , 复用成熟 kernel

图文精读版 | 2025-07-10 | NVIDIA CUDA 重点



解决的问题

block sparse 粒度高效但 token importance 随 layer/head 变化；永久 drop token 会妨碍后续层重新选择。

图中机制

per-head select token subset；compress Q/K/V；在 compact tensors attention；scatter output back 后叠加 residual。

实现 / kernel 含义

kernel 只见连续 compact QKV，可复用 FlashAttention；真实成本为 selector + gather/contiguous + scatter。实现细节为 PDF-only。

长上下文桥接：MInference 的 pattern-aware index 与 kernel dispatch

图文情报版 2024-07-16 NVIDIA CUDA 重点

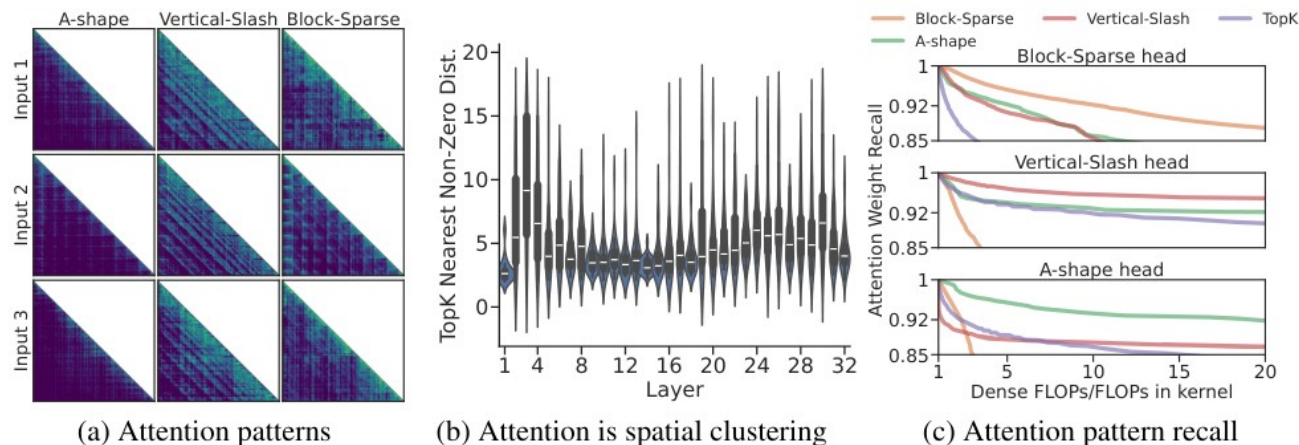


Figure 3: (a) Visualization of attention weights from different attention heads. For different prompts and tasks, the pattern of the same head is relatively consistent, but the sparse indices are dynamically changing. (b) Distance of the top-10 nearest non-zero element in the attention matrix. (c) Attention recall distribution using our identified patterns, where FLOPs in the kernel refer to the real FLOPs required for sparse attention computing using on GPUs. Here, a 1x64 block size is used for the *Vertical-Slash* pattern, and a 64x64 block size is used for others on GPUs. All visualization are based on LLaMA-3-8B-Instruct-262K [Gra24].

解决的问题

prefill attention 延迟主导；固定 top-k index 跨 prompt recall 显著下降。

图中机制

离线给 head 选 A-shape / vertical-slash / block-sparse family；在线建立具体 ranges/columns/blocks。

实现 / kernel 含义

pattern-specific index 交给 PIT/Triton/FlashAttention 类 kernel。迁移到 video 必须改为双向时空 pattern，并处理每 step 的 planner 成本。

架构替代：FrameDiT 以 matrix attention 改变 temporal topology

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

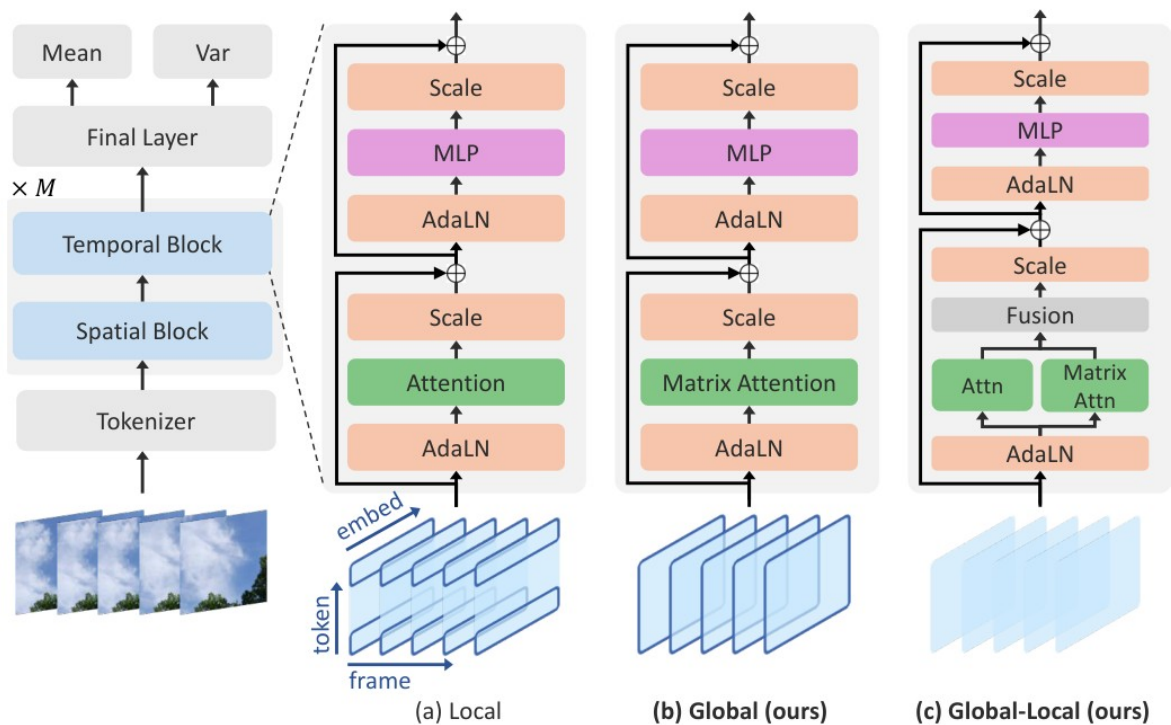


Figure 1. **Overview of the proposed FrameDiT.** Built on the Diffusion Transformer with interleaved Spatial and Temporal blocks (a) Local: conventional local factorized attention; (b) Global (ours): replaces temporal attention with Matrix Attention for frame-level temporal attention; (c) Global-Local Hybrid (ours): combines local and global temporal attention for unified spatio-temporal modeling.

解决的问题

full 3D attention 表达强但昂贵；local factorized attention 便宜却错过大运动。

图中机制

以 frame matrix 为对象做 temporal Matrix Attention，Global-Local hybrid 保留局部路径，不再构造原 token-pair temporal mask。

实现 / kernel 含义

公开代码仍把 2D mask 转为 -10000 dense bias 并 broadcast；论文算法收益不自动等于 custom sparse kernel。

两条“真正下沉到 runtime”的路径：规则与 CSR

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

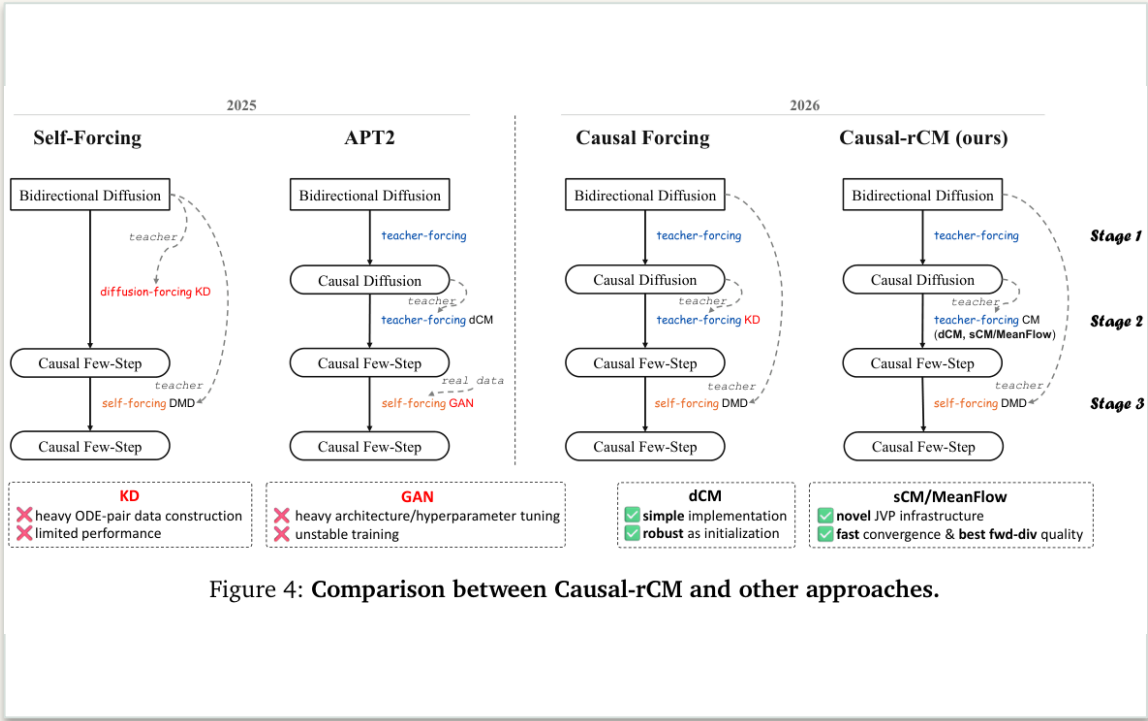


Figure 4: Comparison between Causal-rCM and other approaches.

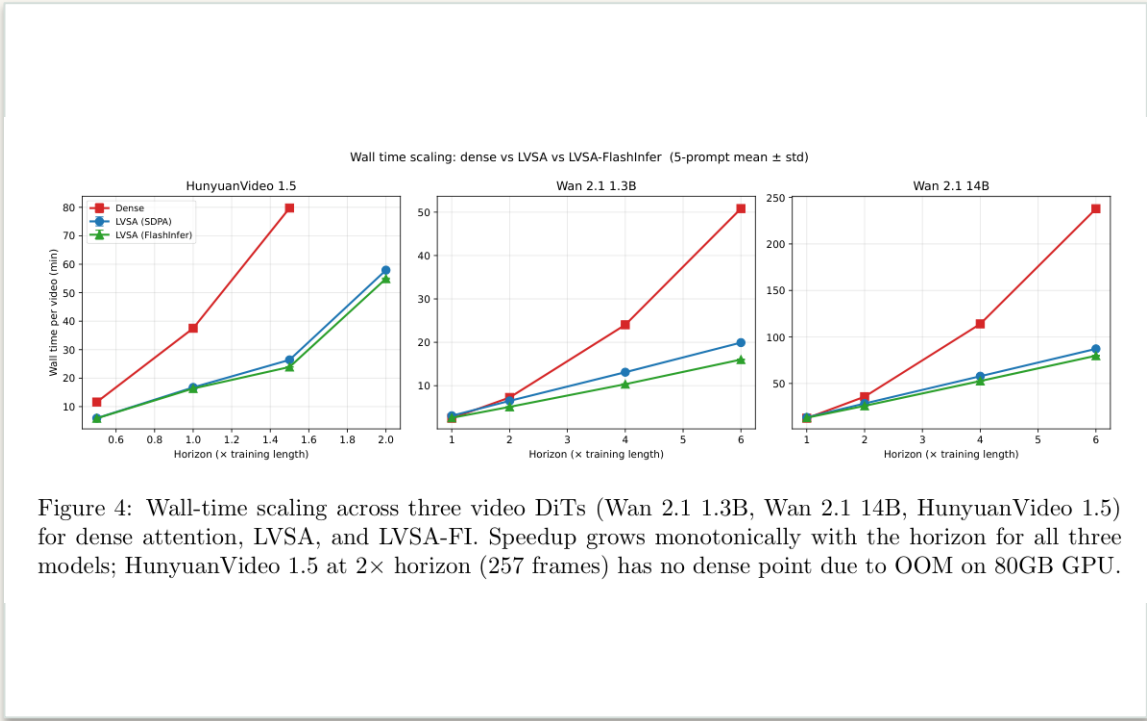


Figure 4: Wall-time scaling across three video DiTs (Wan 2.1 1.3B, Wan 2.1 14B, HunyuanVideo 1.5) for dense attention, LVSA, and LVSA-FI. Speedup grows monotonically with the horizon for all three models; HunyuanVideo 1.5 at 2 $\times$ horizon (257 frames) has no dense point due to OOM on 80GB GPU.

Causal-rCM：规则型

BlockPattern 让 kernel/BlockMask 按 block id 判定 visibility；同一规则覆盖 TF、JVP、cache。

LVSA：索引型

CSR 与 compact layout 交给 FlashInfer planning；图表说明长 horizon/80GB 情况，但不可跨模型横比。

定制 mask 的四种表达：什么能真正跳过 tile ？

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

Rule / BlockMask

block id, window, stream,
offset

Causal-rCM

kernel/compiler 可判定 tile

成本：规则须 **block-aligned**

CSR / page table

indptr, indices, page id

LVSA

scheduler 只遍历 nnz block

成本：**plan / locality**

Selected segments

indices, cu_seqlens, compact
QKV

VMoBA / TSA / VLM

标准 varlen kernel

成本：**pack/unpack**

Dense bias

bool / -inf score bias

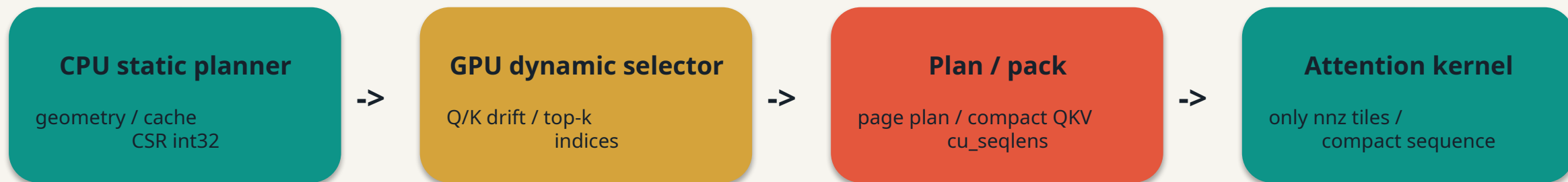
FrameDiT public code

通常不跳 tile

成本： **L^2 / dense work**

长序列 host-device 数据流：传 metadata，不传 dense pair mask

图文讲解版 | 2026-07-10 | NVIDIA CUDA 重点



允许

静态 window/anchor：CPU 一次性 CSR、pinned metadata、FlashInfer plan；每 request 的 page list 也可由 host scheduler 供给。

禁止

CPU 生成 $L \times L$ bool/fp16 mask 再拷 GPU。64K 单 bool mask 已 4GiB；per-step top-k CPU 往返还会同步 pipeline。

建议的实现边界：先 lowering，再选 kernel

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

MaskSemantics: stream partition + temporal geometry + dynamic source

A. rectangles

split into causal/full varlen calls

B. regular graph

BlockMask / predicate

C. explicit graph

CSR/page metadata + plan

D. dynamic selection

gather + compact varlen

```
build_attention_plan(spec, qkv_layout, device) -> Plan
attention_run(q, k, v, Plan) -> out
```

评测公式

$T_{total} = T_{select} + T_{metadata} + T_{H2D/plan} + T_{pack} + T_{attn} + T_{unpack}$ 。只报 attention FLOPs 会遗漏控制面和数据搬运。

结论：让 **mask** 的语义在正确层级消失或变小

图文精读版 | 2026-07-10 | NVIDIA CUDA 重点

1

能拆就拆

双流 / 矩形可见性 -> causal/full varlen calls

2

需稀疏就索引化

window/anchor -> CSR / BlockMask / page table

3

需动态就紧凑化

router/selector -> GPU indices + compact QKV

4

控制面也是性能

planner、H2D、top-k、pack/unpack 必须和 attention 同测